pushdown and bubbleUp will require logarithmic time, as shown in chapter 2, sections 2.6.1 and 2.6.2.

Finding an element in a heap, however, would normally require linear time: but in section 2.6.8 we have shown that it is possible to use an auxiliary hash table inside our priority queue and have search performed in amortized constant time, and thus `updatePriority` run in amortized logarithmic time.

We previously hinted at the fact that we should go easy on using locks, so the question arises: could we acquire the lock later, in the function?

That depends. We need to make sure that while we check the hash table for `key`, no other thread is writing on it. If we use a concurrent structure for the hash table, we could move the write lock acquisition inside the first branch of the if, but then we would also need to change the rest of the function (because evicting one entry and adding the new one should all happen as an atomic operation) and be careful with how we use the hash table, since we introduce a second resource on which we lock, and that might lead to deadlock[31].

So, the short answer is: let's keep it simple (also because we wouldn't have so much of an advantage).

7.7.4 Reentrant Locks

Now we can finally explain how a reentrant lock prevent deadlock in our case. At line marked with #3 in listing 7.10, the current thread calls `evictOneEntry` to remove one item